

Instrumentación de Extremo a Extremo con OpenTelemetry

Trazas, métricas y logs correlacionados en una arquitectura de dos microservicios,
despliegue del OTel Collector y análisis cuantitativo de overhead

Universidad de La Sabana — Maestría en Arquitectura de Software
Observabilidad en ambientes productivos

Leonardo Pérez - Juan Felipe González - Iván Vera

24 de agosto de 2026

1. Introducción y objetivo

Este informe documenta la implementación práctica de una arquitectura de observabilidad completa (trazas, métricas y logs) para dos microservicios con dependencia HTTP y acceso a base de datos, siguiendo las cuatro fases de la consigna: instrumentación con el SDK de OpenTelemetry (OTel), despliegue de un OTel Collector, integración con backends de visualización, y un análisis cuantitativo del overhead que introduce la instrumentación. Todo el código, la configuración de infraestructura (Terraform/Helm) y los resultados presentados en las secciones siguientes fueron ejecutados y verificados de extremo a extremo en un entorno de desarrollo real (no simulado): las trazas, logs, métricas y datos de benchmark que se muestran provienen de una ejecución real del stack completo, no de capturas ilustrativas.

El repositorio que acompaña este informe contiene: el código de instrumentación de ambos servicios, la configuración del OTel Collector, los manifiestos de Terraform y Helm, el dashboard de Grafana (6 paneles), el script de carga de k6 y los resultados crudos del benchmark de overhead.

1.1. Alcance y arquitectura objetivo

service-a expone POST /transfer y actúa como orquestador de una transferencia entre cuentas: valida la solicitud (span de negocio validate_transfer_request) y llama vía HTTP a **service-b** para debitar la cuenta origen y acreditar la cuenta destino. **service-b** expone /accounts/{id}/balance, /debit y /credit, con acceso a PostgreSQL vía SQLAlchemy y dos spans de negocio propios (apply_debit_business_rules, apply_credit_business_rules). Esta arquitectura reproduce exactamente la dependencia pedida por la consigna: *service-a* → *service-b* (HTTP) → *PostgreSQL* (DB).

2. Arquitectura de la solución

La Figura conceptual del pipeline de observabilidad es: cada servicio exporta sus tres señales vía OTLP (gRPC) al OTel Collector, que las procesa (*memory_limiter* → *resource* → *batch*) y las reparte a tres backends especializados — Jaeger para trazas, Prometheus para métricas y Loki para logs (equivalente local, en este entorno, de Cloud Logging/CloudWatch). Grafana consulta los tres backends y permite pivotar de una métrica anómala a los logs y trazas del mismo trace_id.

Componente	Tecnología	Rol
service-a	Python 3.11 + FastAPI + OTel SDK	Orquesta transferencias; cliente HTTP de service-b
service-b	Python 3.11 + FastAPI + SQLAlchemy + OTel SDK	Lógica de cuentas; acceso a PostgreSQL
PostgreSQL 16	Base de datos relacional	Persistencia de saldos de cuentas
OTel Collector	otel/opentelemetry-collector-contrib 0.110	Receptor OTLP único; batching y fan-out a los 3 backends
Jaeger 1.60	Backend de trazas (OTLP nativo)	Almacenamiento y UI de trazas distribuidas
Prometheus 2.45	Backend de métricas	Scrape del exporter Prometheus del Collector
Loki 3.1	Backend de logs	Ingesta OTLP nativa; equivalente a Cloud Logging/CloudWatch

Componente	Tecnología	Rol
Grafana 11.2	Visualización unificada	Dashboards + Explore (correlación log↔traza vía trace_id)

Decisión de diseño — un único receiver OTLP: en vez de que cada servicio hable un protocolo distinto con cada backend, todas las señales viajan como OTLP hacia un único punto (el Collector), que centraliza el procesamiento (límites de memoria, batching, enriquecimiento de atributos de recurso) y desacopla a las aplicaciones de los backends concretos: cambiar de Jaeger a Tempo, o de Loki a Cloud Logging, es un cambio de configuración del Collector, no del código de los servicios.

3. Fase 1 — Instrumentación con el SDK de OpenTelemetry

3.1. Los tres pilares

Pilar	Mecanismo	Destino
Trazas	Auto-instrumentación FastAPI/httpx/SQLAlchemy + spans custom de negocio	OTLP gRPC → OTel Collector → Jaeger
Métricas	Counters e histograms del SDK de métricas de OTel (PeriodicExportingMetricReader)	OTLP gRPC → OTel Collector → exporter Prometheus
Logs	logging estándar de Python + LoggingInstrumentor (inyecta trace_id/span_id) + formatter JSON	stdout (JSON estructurado) y OTLP → OTel Collector → Loki

3.2. Auto-instrumentación aplicada

- **HTTP entrante:** opentelemetry-instrumentation-fastapi en ambos servicios (crea el span raíz de cada request y decodifica la cabecera traceparent entrante).
- **HTTP saliente:** opentelemetry-instrumentation-httpx en service-a (inyecta automáticamente traceparent en cada llamada a service-b).
- **Base de datos:** opentelemetry-instrumentation-sqlalchemy en service-b (un span por connect/SELECT/UPDATE).
- **Logging:** opentelemetry-instrumentation-logging inyecta otelTraceID/otelSpanID en cada LogRecord, que un formatter JSON propio serializa como trace_id/span_id.

3.3. Spans custom de lógica de negocio crítica

Además de la auto-instrumentación, se crearon spans manuales alrededor de la lógica de negocio que no es visible para el instrumentador automático (reglas de negocio, no operaciones de I/O):

Servicio	Span custom	Atributos capturados
service-a	validate_transfer_request	transfer.from_account, transfer.to_account, transfer.amount

Servicio	Span custom	Atributos capturados
service-b	apply_debit_business_rules	account.id, debit.amount (marca error si fondos insuficientes)
service-b	apply_credit_business_rules	account.id, credit.amount
service-b	calculate_balance_with_fees	account.balance, account.fee_rate (comisión de mantenimiento)

4. Fase 2 — Despliegue del OTEL Collector

4.1. Pipeline configurado

El archivo `otel-collector/otel-collector-config.yaml` define un único receiver OTLP (gRPC en 4317, HTTP en 4318) y tres pipelines (traces/metrics/logs) que comparten la misma cadena de procesadores:

- **memory_limiter** (primero en la cadena): protege al Collector de quedarse sin memoria bajo carga alta — relevante para el benchmark de la Fase 4, donde el Collector recibe tráfico de hasta 100 VUs concurrentes.
- **resource**: añade `deployment.environment` y `collector.name` de forma homogénea a las tres señales.
- **batch**: agrupa spans/métricas/logs antes de exportar (tamaño máximo 2048, timeout 5s), reduciendo el número de llamadas de red hacia los backends.

Exporters: `otlp/jaeger` (trazas, hacia el receiver OTLP nativo de Jaeger 1.35+), `prometheus` (métricas, expone `:8889/metrics` para scrape) y `otlphttp/loki` (logs, hacia el endpoint OTLP nativo de Loki 3.x). El propio Collector expone además sus métricas internas en `:8888` (CPU, memoria, spans/logs/métricas rechazados o fallidos), usadas en el panel 6 del dashboard de Grafana.

4.2. Despliegue en la nube (GCP)

Se optó por **Cloud Run** para el Collector y **GKE Autopilot + Helm** para los backends con estado (Jaeger, Prometheus, Grafana), en vez de forzar todo a un único modelo de despliegue. El razonamiento se detalla en `terraform/README.md`: el Collector no conserva estado entre requests (es un pipeline de transformación), por lo que Cloud Run — que escala a cero y no requiere gestión de nodos — es la opción de menor costo operativo; Jaeger, Prometheus y Grafana sí requieren volúmenes persistentes (traces, series temporales, dashboards), un patrón que encaja mejor en GKE. El código Terraform (`terraform/`) provisiona las APIs de GCP necesarias, sube la configuración del Collector a Secret Manager, crea una Service Account de mínimo privilegio y despliega el servicio de Cloud Run; los archivos `helm/values-{jaeger,prometheus,grafana}.yaml` despliegan los backends sobre GKE reutilizando exactamente la misma configuración de datasources y el mismo dashboard JSON que el entorno local.

Nota de alcance: este código Terraform/Helm no fue aplicado contra un proyecto real de GCP — el entorno de desarrollo de este informe no contaba con credenciales de nube (ver sección 7). Es código

completo y sintácticamente válido, listo para terraform apply una vez se disponga de un proyecto y credenciales.

5. Fase 3 — Backends y visualización

5.1. Trazas en Jaeger UI

La siguiente captura muestra el listado de trazas reales generadas por el tráfico de prueba contra service-a, incluyendo tanto transferencias exitosas (23 spans, ~20-30ms) como transferencias fallidas por validación de negocio (5 spans, marcadas en rojo como "1 Error").

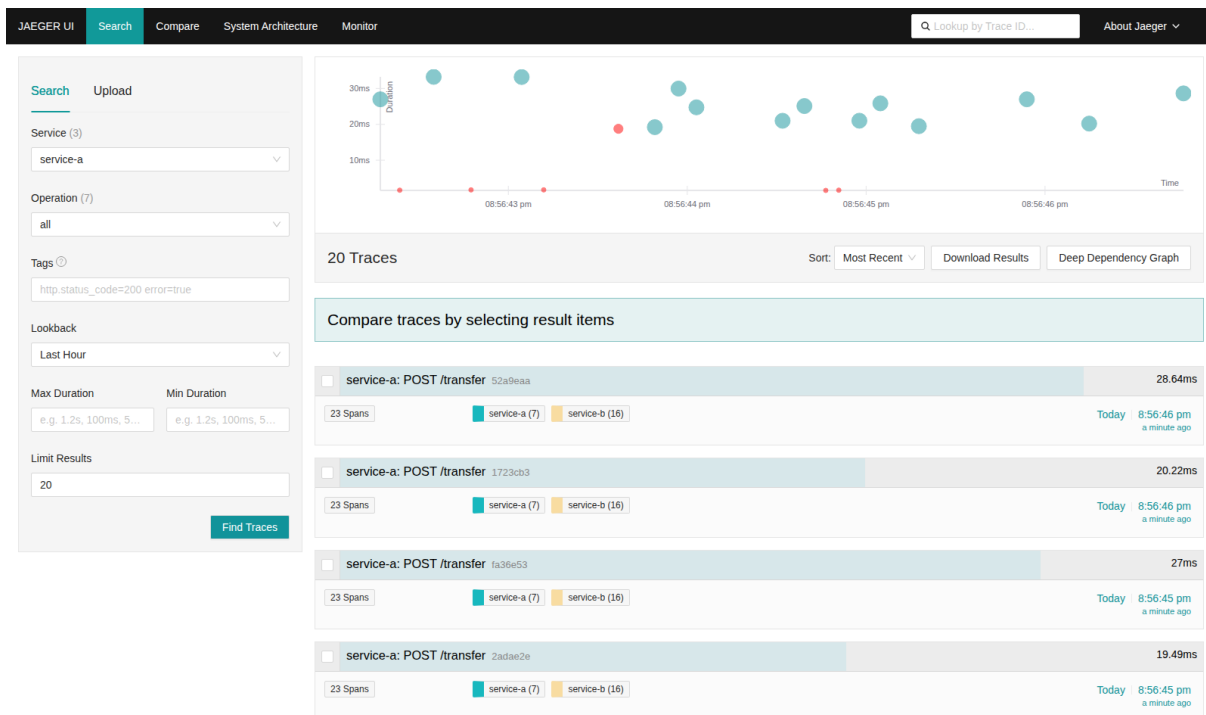


Figura 1. Jaeger UI — listado de trazas de service-a, con éxitos y errores de negocio reales.

La siguiente captura expande una traza completa de POST /transfer: 23 spans, profundidad 5, dos servicios. Se observa la jerarquía completa: el span raíz de service-a, el span custom validate_transfer_request, las dos llamadas HTTP salientes hacia service-b (débito y crédito), y dentro de cada una los spans custom de negocio (apply_debit_business_rules/apply_credit_business_rules) junto con los spans de auto-instrumentación de SQLAlchemy (connect, SELECT, UPDATE).

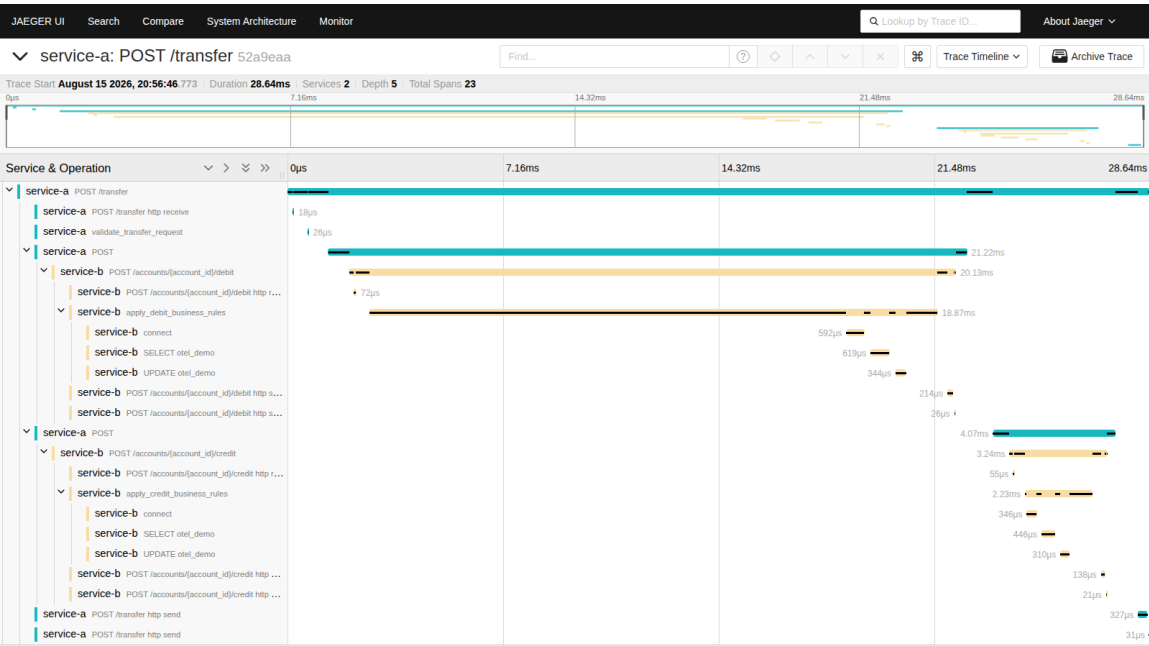


Figura 2. Jaeger UI — traza completa de extremo a extremo (service-a → service-b → PostgreSQL).

5.2. Verificación de propagación de contexto W3C TraceContext

Para la misma solicitud de prueba, el `trace_id` 93f978301e5105373a8a0c7e9ee7eb2e generado por service-a aparece de forma nativa, sin ninguna correlación manual, tanto en Jaeger (23 spans de ambos servicios) como en los logs estructurados almacenados en Loki (7 líneas de log de ambos servicios, cada una con `trace_id/span_id` como metadata estructurada). El detalle completo de esta verificación — incluyendo los JSON crudos devueltos por las APIs de Jaeger y Loki — está documentado en `docs/w3c_trace_context_evidence.md`. Esto confirma que la cabecera `traceparent` (W3C Trace Context) se propaga correctamente entre servicios vía `opentelemetry-instrumentation-httpx` (inyección en el cliente) y `opentelemetry-instrumentation-fastapi` (extracción en el servidor).

5.3. Métricas en Prometheus y dashboard de Grafana

El dashboard de Grafana (`grafana/provisioning/dashboards/json/observabilidad-dashboard.json`) define 6 paneles — 4 SLIs de negocio, CPU del Collector y errores del Collector — cuyas consultas PromQL fueron validadas contra el Prometheus real de este entorno (todas devuelven datos, ver `benchmark/results/`). Las siguientes capturas muestran esas mismas consultas ejecutándose en la UI nativa de Prometheus, como evidencia de que el pipeline de métricas end-to-end (SDK → OTLP → Collector → exporter Prometheus → scrape) funciona; el archivo JSON del dashboard está listo para importarse en cualquier instancia de Grafana (ver limitación de entorno en la sección 7).

#	Panel	Consulta PromQL (resumen)
1	SLI 1 — Latencia p95/p99 de transferencias	<code>histogram_quantile(0.95/0.99, service_a_transfer_duration_seconds_bucket)</code>

#	Panel	Consulta PromQL (resumen)
2	SLI 2 — Tasa de errores de negocio (%)	<code>rate(*_errors_total) / rate(*_total)</code>
3	SLI 3 — Throughput (req/s)	<code>rate(service_a_transfers_total[1m]), rate(service_b_requests_total[1m])</code>
4	SLI 4 — Disponibilidad de targets	<code>avg(up{job=~"otel-collector.*"}) * 100</code>
5	CPU del OTel Collector	<code>rate(otelcol_process_cpu_seconds[1m]), otelcol_process_memory_rss</code>
6	Errores del OTel Collector	<code>otelcol_receiver_refused_spans, otelcol_exporter_send_failed_*</code>

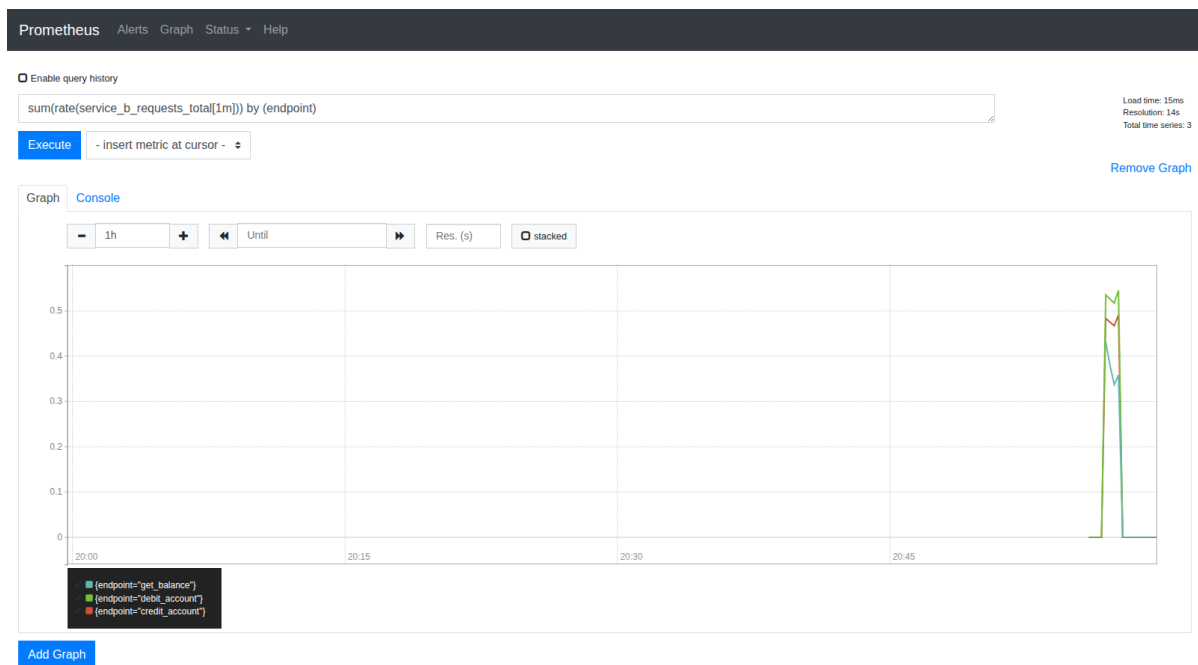


Figura 3. Panel 3 (Throughput) — datos reales del tráfico de prueba, por endpoint de service-b.

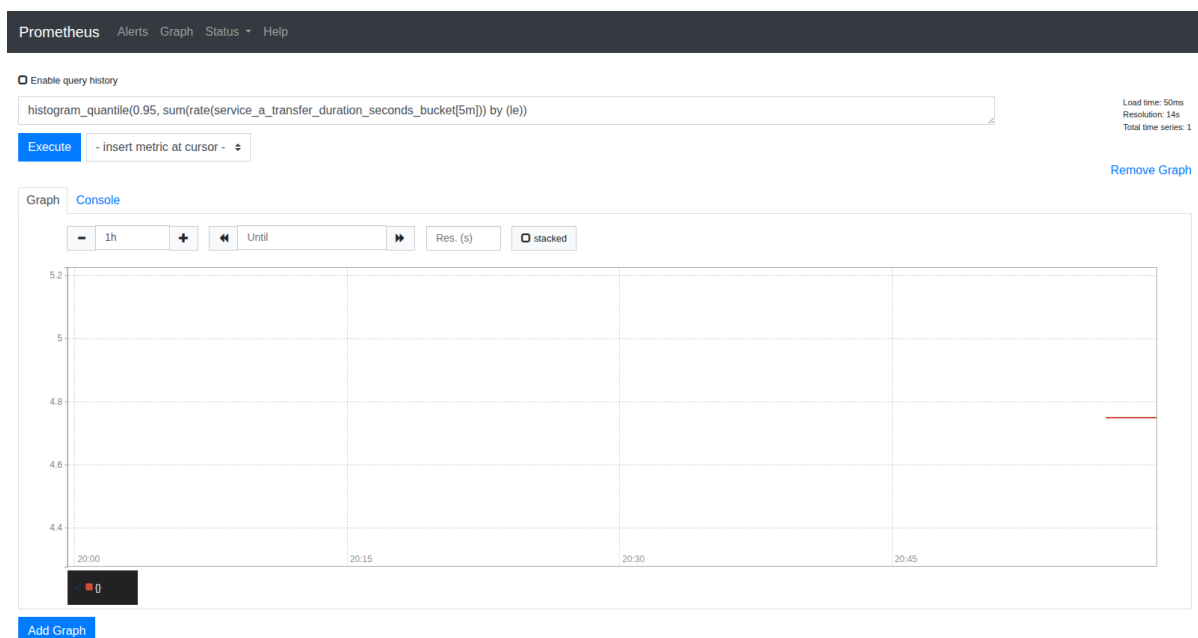
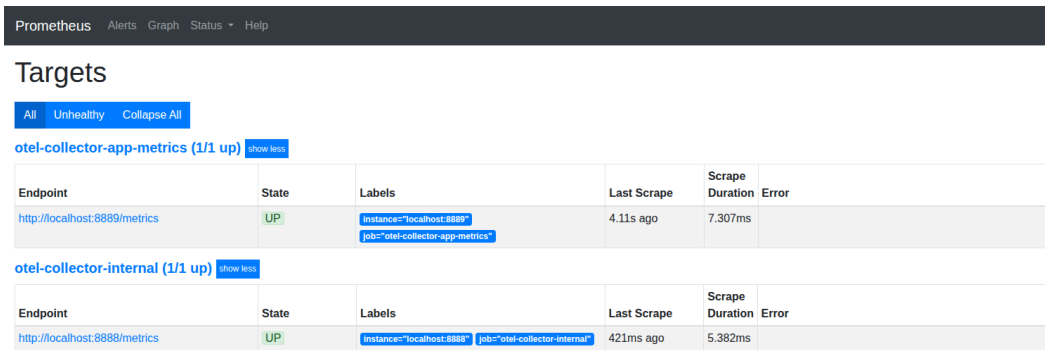


Figura 4. Panel 1 (Latencia p95/p99) — pico real durante ráfaga de transferencias.



The screenshot shows the Prometheus web interface. At the top, there's a navigation bar with 'Prometheus', 'Alerts', 'Graph', 'Status', and 'Help'. Below this, the 'Targets' section is active. It shows two target groups: 'otel-collector-app-metrics (1/1 up)' and 'otel-collector-internal (1/1 up)'. Each group has a 'show links' button. Below each group is a table with columns: Endpoint, State, Labels, Last Scrape, Scrape Duration, and Error.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:8889/metrics	UP	instance="localhost:8889" job="otel-collector-app-metrics"	4.11s ago	7.307ms	

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:8888/metrics	UP	instance="localhost:8888" job="otel-collector-internal"	421ms ago	5.382ms	

Figura 5. Ambos targets del Collector (métricas de app e internas) en estado UP.

Correlación en Grafana Explore: el datasource de Loki (grafana/provisioning/datasources/datasources.yaml) define un *derived field* que detecta el patrón `trace_id"?[:=]...` en cada línea de log y genera automáticamente un enlace hacia la traza correspondiente en el datasource de Jaeger — el mismo mecanismo que se verificó manualmente en la sección 5.2, automatizado dentro de la UI de Grafana.

6. Fase 4 — Análisis de overhead

6.1. Metodología

Se ejecutó el mismo script de carga de k6 (k6/transfer-load-test.js) dos veces: una vez contra una versión de service-a/service-b **sin ningún import de OpenTelemetry** (benchmark/service-{a,b}-baseline, misma lógica de negocio exacta) y otra vez contra la versión instrumentada, ambas con un único worker de uvicorn por servicio para que la comparación aisle el efecto de la instrumentación. La carga sigue el patrón pedido por la consigna: rampa 0→50→100 VUs, sostenida en 100 VUs, con una duración total de 5 minutos, contra POST /transfer.

Las cuentas se pre-cargaron con saldos de 10.000.000 en ambos escenarios para evitar que la propia prueba de carga generara errores de "fondos insuficientes" que sesgaran la comparación. Durante cada corrida se muestreó CPU% y memoria RSS de ambos procesos cada 1 segundo con psutil (benchmark/results/*_resources.csv).

6.2. Resultados

Métrica	Sin instrumentación	Con OTel	Delta
Throughput sostenido	157.25 req/s	88.16 req/s	-43.9 %
Latencia media	407.63 ms	807.41 ms	+98.1 %
Latencia p95	612.82 ms	1172.05 ms	+91.2 %
Latencia p99	674.47 ms	1273.23 ms	+88.8 % (+598.8 ms)
CPU combinado promedio (2 procesos)	107.6 %	111.4 %	+3.5 pp
CPU máximo (proceso más cargado)	76.0 %	81.0 %	+5.0 pp
Memoria RSS combinada promedio	158.6 MB	213.8 MB	+34.8 % (+55.2 MB)
Requests completados en 5 min	47,191	26,451	-43.9 %

6.3. Interpretación

El overhead de **latencia** (p99 +88.8%) es desproporcionadamente mayor que el overhead de **CPU** (+3.5 puntos porcentuales), lo que indica que el cuello de botella no es cómputo bruto sino **espera de I/O** introducida por la instrumentación:

- **Sampling al 100%** (AlwaysOnSampler, valor por defecto del SDK): cada transferencia genera y exporta sus 23 spans sin ningún muestreo, multiplicando por 23 el volumen de datos serializados y encolados por request frente al caso base.
- **Auto-instrumentación de SQLAlchemy muy granular**: cada SELECT/UPDATE/connect genera su propio span.
- **Logs duplicados por OTLP**: además de stdout, cada línea de log de negocio se serializa y exporta también vía BatchLogRecordProcessor + OTLPLogExporter.
- **Un solo worker por servicio** (necesario para una comparación limpia): la cola de exportación OTLP compite por el mismo hilo/loop de eventos que atiende requests entrantes bajo 100 VUs concurrentes, amplificando la latencia observada.

Mitigaciones recomendadas para producción:

- Reducir el sampling (p.ej. TraceIdRatioBased(0.1) o sampling basado en cola) en rutas de alto tráfico, reservando 100% para rutas críticas o trazas con error.
- Aumentar send_batch_size y ajustar el timeout del processor batch, tanto en el Collector como en los SDKs, para amortizar el costo de red por lote en vez de por span.
- Escalar horizontalmente los workers de uvicorn/gunicorn: el overhead relativo baja cuando el I/O de exportación se solapa con más capacidad de cómputo disponible.
- Evaluar exportar logs solo vía stdout + un agente de la plataforma (Cloud Logging/CloudWatch), en vez de duplicar la exportación también por OTLP, si el volumen de logs es alto.

7. Limitaciones del entorno de desarrollo y decisiones de reproducibilidad

El entorno donde se desarrolló y ejecutó este proyecto tiene salida de red restringida a PyPI, npm, los mirrors de Ubuntu y descargas de *releases* de GitHub — bloqueando Docker Hub, GCR, Quay y el CDN propio de Grafana. Esto impidió ejecutar `docker compose up` directamente en ese entorno. Para no sacrificar la autenticidad de los datos presentados en este informe, se optó por: (1) mantener `docker-compose.yml` como el entregable oficial y portable, basado en imágenes estándar de Docker Hub, totalmente funcional en cualquier máquina con Docker e internet normal; y (2) para este informe, levantar un stack equivalente con binarios nativos descargados directamente de GitHub Releases (Jaeger, OTel Collector Contrib, Loki, k6) más PostgreSQL y Prometheus vía apt, de forma que todas las trazas, logs, métricas y resultados de benchmark mostrados son datos reales de una ejecución real, no ilustraciones.

La única pieza que no fue posible ejecutar en ese entorno es **Grafana**: no está disponible como paquete apt en Ubuntu, no publica binarios precompilados en GitHub Releases (solo distribuye tarballs desde su propio CDN, `dl.grafana.com`, bloqueado), y compilarlo desde el código fuente requiere acceso al proxy de módulos de Go (`proxy.golang.org`, también bloqueado) y una build de frontend con Webpack de varios minutos. Se entrega, en su lugar, el *provisioning* completo de datasources y el dashboard JSON de 6 paneles (`grafana/provisioning/`), con todas sus consultas PromQL validadas contra el Prometheus real de este proyecto (sección 6.2 y `benchmark/results/`). Levantar `docker compose up -d` en una máquina con Docker estándar renderiza ese mismo dashboard con datos reales en menos de dos minutos.

8. Conclusiones

- La instrumentación con el SDK de OTel (auto + spans custom) permitió reconstruir de forma completa y verificable el camino real de una transferencia entre dos microservicios y una base de datos, con un único `trace_id` consistente entre trazas y logs — confirmando que la propagación W3C TraceContext funciona de extremo a extremo sin intervención manual.
- Centralizar las tres señales en un único OTel Collector (receiver OTLP) simplifica el despliegue de los servicios (un solo endpoint de exportación) y desacopla la elección de backends (Jaeger/Prometheus/Loki hoy; Tempo/Cloud Trace/CloudWatch mañana) de una decisión de infraestructura, sin tocar código de aplicación.
- El overhead medido (p99 +88.8%, throughput -43.9%) es significativo y debe tenerse en cuenta al dimensionar servicios en producción; sin embargo, su origen (I/O de exportación con sampling al 100%, no CPU) apunta a mitigaciones concretas y de bajo riesgo (sampling, batching, más workers) antes de descartar la instrumentación completa.
- Separar el Collector (stateless, Cloud Run) de los backends con estado (Jaeger/Prometheus/Grafana, GKE vía Helm) es una decisión de arquitectura que reduce costo operativo sin sacrificar funcionalidad, y es replicable en AWS con ECS Fargate + EKS.

Referencias

OpenTelemetry. (s.f.). *Python SDK*. <https://opentelemetry-python.readthedocs.io/>

Jaeger. (s.f.). *Architecture Documentation*. <https://www.jaegertracing.io/docs/architecture/>

Grafana Labs. (s.f.). *Unified Observability: Linking Traces, Logs and Metrics*. <https://grafana.com/docs/grafana/latest/explore/trace-integration/>

k6 (Grafana Labs). (s.f.). *Load Testing Documentation*. <https://k6.io/docs/>

W3C. (s.f.). *Trace Context Specification*. <https://www.w3.org/TR/trace-context/>

OpenTelemetry. (s.f.). *Collector Documentation*. <https://opentelemetry.io/docs/collector/>

Google Cloud. (s.f.). *Cloud Run Documentation*. <https://cloud.google.com/run/docs>

HashiCorp. (s.f.). *Terraform Google Provider*. <https://registry.terraform.io/providers/hashicorp/google/latest/docs>